Table 8-41. *intl OOP APIs*

`Collator`	Sorting functionality
`NumberFormatter`	Format numbers, including currency values
`Locale`	Manage regional settings (system dependent)
`Normalizer`	Transform characters for sorting and searching
`MessageFormatter`	Generate messages from fragments with localized structure
`IntlCalendar`	Manipulate and localize dates and time
`IntlGregorianCalendar`	Date and time localization, using the Gregorian system
`IntlTimeZone`	Manage regional time
`IntlDateFormatter`	Create dates regionally formatted for display
`ResourceBundle`	Create simple or complex regional data objects
`Transliterator`	Create conversion rules for case transformation, character-set transforms, etc.
`IntlBreakIterator`	Locate word boundaries in text
`IntlRuleBasedIterator`	Sub-class of `IntlBreakIterator` with custom rules
`UConverter`	Converts between character sets
`grapheme functions`	Set of functions that handles strings based on grapheme width (e.g., multi-codepoint glyphs)
`IDN functions`	Convert domain names to IDNA (ASCII or UTF-8)
`IntlChar`	Utilities for obtaining metadata for Unicode characters
`IntlIterator`	Multi-byte character-aware iteration
`intl functions`	Set of error-handling functions for the `intl` extension

A full analysis of the `intl` extension would cover at least a chapter in its own right, and we've already covered enough ground that it would be repeating a lot of what has already been shown. I highly recommend taking a look at the official documentation though; there are a lot of useful tools that will save you from having to roll your own code: https://www.php.net/manual/en/book.intl.php